Universidad Tecmilenio

Gestión de proyectos de inteligencia artificial



Docente: *Eliskar Roger Altamirano Pariente*

Estudiante: *Páez García Emilio* ***AL07183058***

Actividad 4. *Modelos Preentrenados de Hugging Face*

Fecha de entrega: *11 de oct. del 2026*



Introducción

El análisis de sentimientos se ha convertido en una herramienta fundamental para las organizaciones que buscan comprender la percepción de sus clientes a partir de comentarios y opiniones publicadas en medios digitales. En el sector alimenticio, la capacidad de identificar automáticamente opiniones positivas, negativas o neutrales permite conocer fortalezas, detectar áreas de mejora y apoyar la toma de decisiones basada en datos.

Gracias a los avances en procesamiento de lenguaje natural (PLN), actualmente existen modelos preentrenados que permiten realizar este tipo de análisis con altos niveles de precisión sin necesidad de desarrollar soluciones desde cero, únicamente dependiendo de los datos recopilados a lo largo de la historia de alguna cadena de restaurantes o de algún establecimiento.

En esta actividad se desarrolló un sistema para comparar el desempeño de tres modelos preentrenados disponibles en Hugging Face: RoBERTuito, DistilBERT Multilingüe y XLM-RoBERTa. Para ello, se utilizó un conjunto de opiniones de restaurantes que fue previamente validado y depurado para garantizar la calidad de los datos. Posteriormente, se ejecutaron procesos de inferencia, normalización de etiquetas y evaluación mediante métricas de clasificación como accuracy, precision, recall y F1-score. Asimismo, se analizaron los tiempos de procesamiento de cada modelo con el propósito de evaluar no solo su capacidad predictiva, sino también su eficiencia computacional.

El objetivo principal consistió en identificar cuál de los modelos ofrece el mejor equilibrio entre calidad de clasificación y velocidad de inferencia para tareas de análisis de sentimientos en opiniones de clientes y así poder determinar que modelo se podría adaptar mejor a un caso de uso real, donde un restaurante o establecimiento quiera analizar a profundidad la percepción de la comida, lugar, tiempos de sus clientes



Desarrollo

Preparación del entorno y dataset

Para el comienzo de esta práctica, primero se deberán importar las bibliotecas necesarias para desarrollar el sistema de análisis de sentimientos. Se utilizan herramientas para manipular los datos, ejecutar los modelos preentrenados de Hugging Face, calcular métricas y generar gráficas. También se establece una semilla para favorecer la reproducibilidad y se identifica si el procesamiento se realizará mediante CPU o GPU.

```
# Importaciones y configuración

import os
import time
import random
import numpy as np
import pandas as pd
import torch
import matplotlib.pyplot as plt
import seaborn as sns

from transformers import pipeline
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)
from IPython.display import display
```

```
# Reproducibilidad
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

# Selección automática del dispositivo
device = 0 if torch.cuda.is_available() else -1

print("Versión de PyTorch:", torch.__version__)
print("Versión de CUDA disponible:", torch.version.cuda)
print("GPU disponible:", torch.cuda.is_available())
print("Dispositivo:", torch.cuda.get_device_name(0)
      if torch.cuda.is_available() else "CPU")
```

0.0s

```
Versión de PyTorch: 2.12.1+cpu
Versión de CUDA disponible: None
GPU disponible: False
Dispositivo: CPU
```

Imagen 1.- Preparando el entorno de trabajjo

Se carga el archivo dataset_simulado_opiniones.csv mediante Pandas. Los datos se almacenan en un DataFrame llamado df, que permite consultar y manipular las opiniones de los clientes. Finalmente, se muestran las primeras filas y las dimensiones del dataset para comprobar que la información se haya cargado correctamente.

```
# Carga y validación del dataset

RUTA_DATASET = "dataset_simulado_opiniones.csv"

df = pd.read_csv(RUTA_DATASET)

print("Dimensiones del dataset:", df.shape)
print("\nColumnas:", df.columns.tolist())

display(df.head())
```

0.0s

```
Dimensiones del dataset: (250, 5)

Columnas: ['fecha', 'sentimiento', 'aspecto', 'comentario', 'calificacion']
```

	fecha	sentimiento	aspecto	comentario	calificacion
0	2025-11-08	positivo	tiempo de espera	La tiempo de espera estuvo excelente, definiti...	4
1	2025-11-05	positivo	ambiente	Muy buena ambiente, vale totalmente la pena. (...	4
2	2025-11-05	neutral	tiempo de espera	La tiempo de espera estuvo bien, nada fuera de...	3
3	2025-11-05	negativo	comida	La comida dejó mucho que desear, no volvería. ...	2
4	2025-11-14	positivo	comida	La comida estuvo excelente, definitivamente vo...	5



Imagen 2.- Carga del dataset a utilizar

Se comprueba que el dataset contenga las columnas necesarias para el análisis, principalmente comentario y sentimiento. Después, elimina los registros incompletos, limpia los textos y unifica las etiquetas de sentimiento en tres categorías: NEG, NEU y POS.También verifica que todas las etiquetas sean reconocidas y que existan comentarios válidos para procesar. El objetivo es garantizar que los datos tengan un formato consistente y evitar errores durante la ejecución de los modelos y el cálculo de las métricas.

```python
# Validación y limpieza de los datos

columnas_requeridas = {"comentario", "sentimiento"}

faltantes = columnas_requeridas - set(df.columns)

if faltantes:
    raise ValueError(f"Faltan columnas obligatorias: {faltantes}")

# Eliminar comentarios o etiquetas vacíos
df = df.dropna(subset=["comentario", "sentimiento"]).copy()

df["comentario"] = df["comentario"].astype(str).str.strip()
df["sentimiento"] = (
    df["sentimiento"].astype(str).str.strip().str.lower()
)

# Eliminar comentarios vacíos
df = df[df["comentario"] != ""].copy()

# Convertir las etiquetas originales a un formato uniforme
mapeo_reales = {
    "negativo": "NEG",
    "negative": "NEG",
    "neg": "NEG",
    "neutral": "NEU",
    "neu": "NEU",
    "positivo": "POS",
    "positive": "POS",
    "pos": "POS"
}

df["label_real"] = df["sentimiento"].map(mapeo_reales)

if df["label_real"].isna().any():
    etiquetas_desconocidas = df.loc[
        df["label_real"].isna(), "sentimiento"
    ].unique()
    raise ValueError(
        f"Etiquetas no reconocidas: {etiquetas_desconocidas}"
    )
```

```python
if len(df) == 0:
    raise ValueError("No hay comentarios válidos para evaluar.")

df = df.reset_index(drop=True)

print("Comentarios válidos:", len(df))
print("\nDistribución de sentimientos:")
display(df["label_real"].value_counts().rename("cantidad").to_frame())

print("\nEjemplos:")
display(df[["comentario", "sentimiento", "label_real"]].head())
```

✓ 0.0s

```
Comentarios válidos: 250

Distribución de sentimientos:
```

label_real	cantidad
POS	144
NEU	61
NEG	45

```
Ejemplos:
```

	comentario	sentimiento	label_real
0	La tiempo de espera estuvo excelente, definiti...	positivo	POS
1	Muy buena ambiente, vale totalmente la pena. (...	positivo	POS
2	La tiempo de espera estuvo bien, nada fuera de...	neutral	NEU
3	La comida dejó mucho que desear, no volvería. ...	negativo	NEG
4	La comida estuvo excelente, definitivamente vo...	positivo	POS

Imagen 3.- Validación y limpieza de datos

En esta celda se contabilizan las opiniones negativas, neutrales y positivas, y se genera una gráfica de barras para representar su distribución. Esto permite conocer qué categorías predominan en el conjunto de datos y si existe un desequilibrio importante entre ellas. Por ejemplo, si hay muchas más opiniones positivas que negativas, un modelo podría clasificar correctamente la mayoría de los comentarios,



pero tener dificultades para identificar las quejas. El objetivo es comprender las características del dataset antes de evaluar los modelos y determinar si es necesario prestar especial atención a alguna categoría durante la comparación.

```
# Visualizar la distribución de clases

orden = ["NEG", "NEU", "POS"]

conteo = (
    df["label_real"]
    .value_counts()
    .reindex(orden, fill_value=0)
)

plt.figure(figsize=(7, 4))
conteo.plot(kind="bar")
plt.title("Distribución de sentimientos en el dataset")
plt.xlabel("Sentimiento real")
plt.ylabel("Número de comentarios")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

2] ✓ 0.1s

.

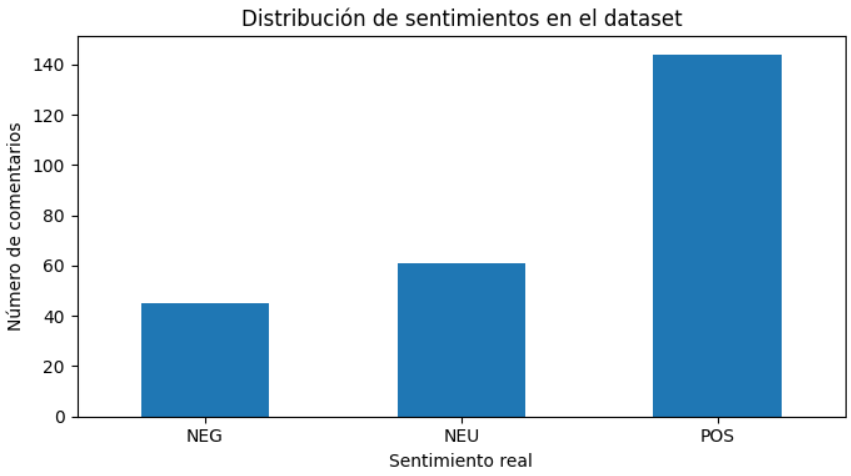


Imagen 4.- Visualización de la distribución de clases

Se establecen los tres modelos preentrenados que se utilizarán en el experimento: RoBERTuito, DistilBERT multilingüe y XLM-RoBERTa. Cada uno se identifica mediante su nombre y la dirección de su repositorio en Hugging Face. La selección permite comparar modelos con diferentes enfoques para el análisis de sentimientos, considerando aspectos como su especialización en español, su capacidad multilingüe y su eficiencia. El objetivo es definir los modelos que



participarán en el experimento para determinar cuál ofrece mejores resultados al clasificar opiniones de restaurantes.

```python
# Definición de los modelos

MODELOS = {
    "RoBERTuito": (
        "pysentimiento/robertuito-sentiment-analysis"
    ),
    "DistilBERT multilingüe": (
        "lxyuan/distilbert-base-multilingual-cased-sentiments-student"
    ),
    "XLM-RoBERTa": (
        "cardiffnlp/twitter-xlm-roberta-base-sentiment"
    )
}

print("Modelos que se evaluarán:")
for nombre, identificador in MODELOS.items():
    print(f"- {nombre}: {identificador}")
```

```
3] ✓ 0.0s

Modelos que se evaluarán:
- RoBERTuito: pysentimiento/robertuito-sentiment-analysis
- DistilBERT multilingüe: lxyuan/distilbert-base-multilingual-cased-sentiments-student
- XLM-RoBERTa: cardiffnlp/twitter-xlm-roberta-base-sentiment
```

Imagen 5.- Definición de los modelos

Carga y normalización de modelos

Se cargan los modelos mediante la función ***pipeline()*** de Transformers, que facilita la clasificación de texto sin necesidad de implementar manualmente todo el proceso de inferencia. Una vez cargado cada modelo, se consultan las etiquetas que utiliza para identificar los sentimientos. Además, se registran los posibles errores de carga, como problemas de conexión, dependencias o incompatibilidades.

```python
# Carga de modelos y revisión de sus etiquetas

clasificadores = {}
errores_carga = {}

for nombre, identificador in MODELOS.items():
    print(f"\nCargando {nombre}...")

    try:
        clasificador = pipeline(
            task="text-classification",
            model=identificador,
            device=device
        )

        clasificadores[nombre] = clasificador

        print("Etiquetas:", clasificador.model.config.id2label)

    except Exception as e:
        errores_carga[nombre] = str(e)
        print(f"Error al cargar {nombre}: {e}")

if not clasificadores:
    raise RuntimeError(
        "No se pudo cargar ningún modelo. "
        "Revisa la conexión, las dependencias y los errores anteriores."
    )

print("\nModelos cargados:", list(clasificadores.keys()))
```

```
✓ 6m 51.4s                                                    Python

Cargando RoBERTuito...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Etiquetas: {0: 'NEG', 1: 'NEU', 2: 'POS'}

Cargando DistilBERT multilingüe...

c:\Users\haaal\anaconda3\envs\FaceIT\Lib\site-packages\huggingface_hub\file_download.py:138: UserWarning: `huggingface_hub` cache-system uses symlinks by default to efficiently store duplicated files but your machine does not support them in C:\Users\haaal\.c
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to activate developer mode, see this article: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-developmen
  warnings.warn(message)

Etiquetas: {0: 'positive', 1: 'neutral', 2: 'negative'}

Cargando XLM-RoBERTa...

c:\Users\haaal\anaconda3\envs\FaceIT\Lib\site-packages\huggingface_hub\file_download.py:138: UserWarning: `huggingface_hub` cache-system uses symlinks by default to efficiently store duplicated files but your machine does not support them in C:\Users\haaal\.c
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to activate developer mode, see this article: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-developmen
  warnings.warn(message)

Etiquetas: {0: 'negative', 1: 'neutral', 2: 'positive'}

Modelos cargados: ['RoBERTuito', 'DistilBERT multilingüe', 'XLM-RoBERTa']
```

Imagen 6.- Carga de modelos y revisión de sus etiquetas



Esta celda contiene una función que convierte las etiquetas generadas por los modelos a un formato común. Por ejemplo, las etiquetas POSITIVE, POSITIVO y POS se convierten en POS. También contempla los casos en los que un modelo utiliza identificadores genéricos, como LABEL_0, consultando su configuración para determinar la categoría correspondiente. El objetivo es unificar las salidas de los modelos y asegurar que las predicciones puedan compararse correctamente con las etiquetas reales del dataset.

```python
# Inferencia y medición del modelo

def normalizar_prediccion(etiqueta, clasificador):
    etiqueta_original = str(etiqueta).strip()
    etiqueta = etiqueta_original.upper()

    equivalencias = {
        "NEG": "NEG",
        "NEGATIVE": "NEG",
        "NEGATIVO": "NEG",
        "NEU": "NEU",
        "NEUTRAL": "NEU",
        "NEUTRO": "NEU",
        "POS": "POS",
        "POSITIVE": "POS",
        "POSITIVO": "POS"
    }

    if etiqueta in equivalencias:
        return equivalencias[etiqueta]

    # Resolver etiquetas genéricas como LABEL_0
    if etiqueta.startswith("LABEL_"):
        try:
            indice = int(etiqueta.split("_")[-1])
            etiqueta_config = clasificador.model.config.id2label.get(
                indice,
                clasificador.model.config.id2label.get(str(indice), "")
            )
            return normalizar_prediccion(etiqueta_config, clasificador)
        except (ValueError, TypeError):
            pass

    raise ValueError(
        f"No se reconoce la etiqueta '{etiqueta_original}'. "
        "Revisa la configuración del modelo."
    )
```

✓ 0.0s

Imagen 7.- Inferencia y medición del modelo

Inferencia y evaluación

En esta celda se ejecutan los tres modelos sobre la misma lista de comentarios. Cada modelo genera una predicción de sentimiento para cada opinión, y los resultados se almacenan para utilizarlos en las siguientes etapas. Además, se mide el tiempo total de inferencia y se calcula el tiempo promedio necesario para procesar un comentario.



También se estima cuántos comentarios puede procesar cada modelo por segundo. El objetivo es obtener las predicciones que se utilizarán para evaluar la calidad de clasificación y medir la velocidad de procesamiento de cada modelo bajo las mismas condiciones.

```
Evaluando RoBERTuito...
Comentarios procesados: 250
Tiempo total: 4.33 s
Tiempo promedio: 0.0173 s/comentario

Evaluando DistilBERT multilingüe...
Comentarios procesados: 250
Tiempo total: 2.16 s
Tiempo promedio: 0.0086 s/comentario

Evaluando XLM-RoBERTa...
Comentarios procesados: 250
Tiempo total: 4.02 s
Tiempo promedio: 0.0161 s/comentario
```

	Modelo	Tiempo total (s)	Tiempo promedio (s)	Comentarios por segundo
0	RoBERTuito	4.3319	0.0173	57.7108
1	DistilBERT multilingüe	2.1568	0.0086	115.9106
2	XLM-RoBERTa	4.0178	0.0161	62.2231

Imagen 8.- Ejecución de los 3 modelos en una sola celda

Se comparan las etiquetas predichas por cada modelo con los sentimientos reales del dataset. Para ello, calcula cuatro métricas principales:

- Accuracy: mide la proporción total de comentarios clasificados correctamente.
- Precision: indica qué tan fiables son las predicciones de cada categoría.
- Recall: mide la capacidad del modelo para identificar los comentarios que realmente pertenecen a cada categoría.
- F1-score: combina precision y recall en una sola medida.

Las métricas se calculan mediante promedios macro y weighted para analizar tanto el rendimiento equilibrado entre las clases como el desempeño considerando su distribución.

✓ 0.0s

	Modelo	Accuracy	Precision macro	Recall macro	F1 macro	Precision weighted	Recall weighted	F1 weighted
0	XLM-RoBERTa	0.852	0.9106	0.7978	0.8064	0.8779	0.852	0.8269
1	DistilBERT multilingüe	0.820	0.9147	0.7541	0.7574	0.8613	0.820	0.7791
2	RoBERTuito	0.768	0.9043	0.6831	0.6420	0.8346	0.768	0.6823

Imagen 9.- Cálculo de métricas de clasificación



Se genera un informe de clasificación individual para cada modelo. A diferencia de la celda anterior, que presenta una tabla comparativa general, aquí se muestran los resultados específicos para las categorías negativa, neutral y positiva. El informe permite observar si un modelo tiene dificultades particulares con alguna clase. Por ejemplo, puede obtener buenos resultados generales, pero presentar un recall bajo para las opiniones negativas.

```
============================================================
INFORME DE CLASIFICACIÓN: RoBERTuito
============================================================
              precision    recall  f1-score   support

    Negativo     1.0000    1.0000    1.0000        45
     Neutral     1.0000    0.0492    0.0938        61
    Positivo     0.7129    1.0000    0.8324       144

    accuracy                         0.7680       250
   macro avg     0.9043    0.6831    0.6420       250
weighted avg     0.8346    0.7680    0.6823       250

============================================================
INFORME DE CLASIFICACIÓN: DistilBERT multilingüe
============================================================
              precision    recall  f1-score   support

    Negativo     0.9783    1.0000    0.9890        45
     Neutral     1.0000    0.2623    0.4156        61
    Positivo     0.7660    1.0000    0.8675       144

    accuracy                         0.8200       250
   macro avg     0.9147    0.7541    0.7574       250
weighted avg     0.8613    0.8200    0.7791       250

============================================================
INFORME DE CLASIFICACIÓN: XLM-RoBERTa
============================================================
              precision    recall  f1-score   support

    Negativo     0.9184    1.0000    0.9574        45
     Neutral     1.0000    0.3934    0.5647        61
    Positivo     0.8136    1.0000    0.8972       144

    accuracy                         0.8520       250
   macro avg     0.9106    0.7978    0.8064       250
weighted avg     0.8779    0.8520    0.8269       250
```

Imagen 10.- Informe detallado por modelo

Esta celda genera una matriz de confusión para cada modelo mediante las etiquetas reales y las predicciones obtenidas. Las matrices se representan como mapas de calor para facilitar su interpretación visual. La diagonal principal muestra las clasificaciones correctas, mientras que las celdas restantes representan errores. De esta forma, es posible identificar si un modelo confunde con frecuencia las opiniones neutrales con las positivas o las negativas. El objetivo es visualizar los patrones de error de cada modelo y comprender con mayor claridad qué tipos de opiniones presentan mayores dificultades de clasificación.

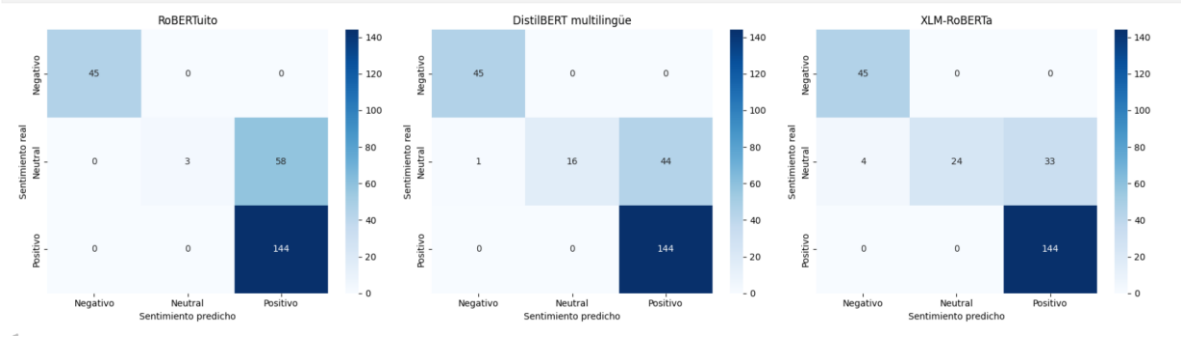


Imagen 11.- Matrices de confusión



Comparación y análisis de errores

En esta celda se construye una gráfica de barras que reúne las métricas principales de los tres modelos: accuracy, precision macro, recall macro y F1 macro. La representación visual facilita identificar las diferencias de rendimiento sin tener que revisar cada valor individualmente. También permite observar si un modelo presenta un desempeño más equilibrado entre las distintas métricas. El objetivo es comparar visualmente la calidad de clasificación de los modelos y facilitar la interpretación de los resultados obtenidos en la evaluación.

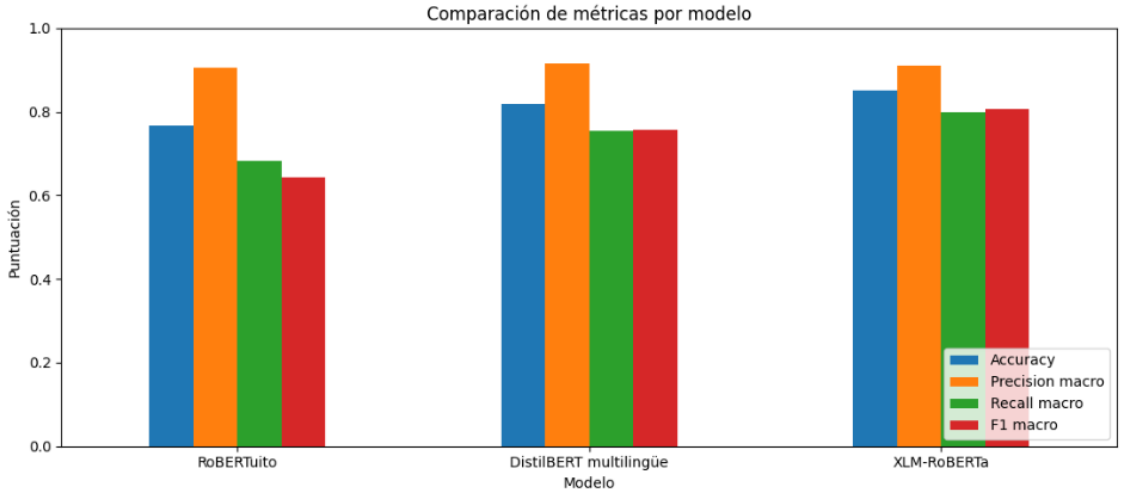
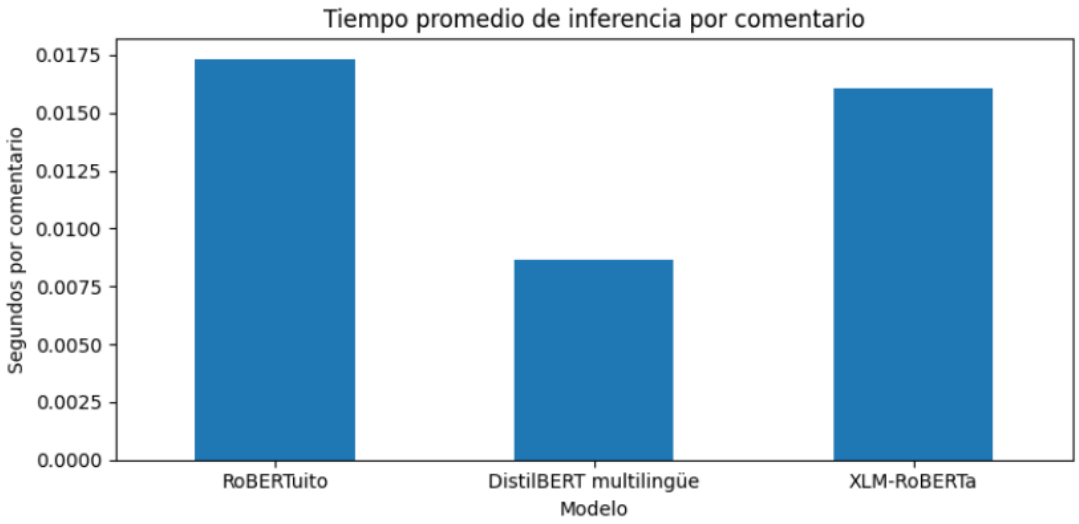


Imagen 12.- Comparación gráfica de métricas

Esta celda utiliza los tiempos registrados durante la ejecución de los modelos para generar una gráfica comparativa del tiempo promedio por comentario. Un menor tiempo de inferencia indica que el modelo procesa las opiniones más rápidamente en las condiciones evaluadas. Esto resulta importante cuando se pretende analizar una gran cantidad de reseñas o integrar el sistema en una aplicación. Es importante considerar que el tiempo depende del hardware, del tamaño de los lotes y de las características de los comentarios. El objetivo es evaluar la eficiencia de los modelos y determinar cuáles ofrecen una mayor velocidad de procesamiento, complementando los resultados de calidad obtenidos mediante las métricas de clasificación.





	Modelo	Tiempo total (s)	Tiempo promedio (s)	Comentarios por segundo
1	DistilBERT multilingüe	2.1568	0.0086	115.9106
2	XLM-RoBERTa	4.0178	0.0161	62.2231
0	RoBERTuito	4.3319	0.0173	57.7108

Imagen 13.- Comparación de tiempos de inferencia

Se combinan las tablas de métricas y tiempos de inferencia para construir una comparación global. Posteriormente, los modelos se ordenan según su F1-score macro, dando prioridad al valor más alto y utilizando el tiempo promedio como criterio secundario. El nombre del modelo que ocupa la primera posición se almacena en la variable mejor_modelo. Esta selección sigue los criterios definidos para el experimento; sin embargo, el resultado debe interpretarse considerando las necesidades del problema. Por ejemplo, para detectar quejas de clientes podría ser especialmente importante que el modelo identifique correctamente las opiniones negativas.

	Modelo	Accuracy	Precision macro	Recall macro	F1 macro	Precision weighted	Recall weighted	F1 weighted	Tiempo total (s)	Tiempo promedio (s)	Comentarios por segundo
0	XLM-RoBERTa	0.852	0.9106	0.7978	0.8064	0.8779	0.852	0.8269	4.0178	0.0161	62.2231
1	DistilBERT multilingüe	0.820	0.9147	0.7541	0.7574	0.8613	0.820	0.7791	2.1568	0.0086	115.9106
2	RoBERTuito	0.768	0.9043	0.6831	0.6420	0.8346	0.768	0.6823	4.3319	0.0173	57.7108

```
Mejor modelo según F1 macro: XLM-RoBERTa
F1 macro: 0.8064
Tiempo promedio: 0.0161 segundos por comentario
```

Imagen 14.-Selección del mejor modelo en base a las métricas



Esta última celda reúne las predicciones de los tres modelos junto con información adicional del dataset, como el comentario, el aspecto evaluado y la calificación, cuando estas columnas están disponibles. Después, identifica las opiniones para las que los modelos generan predicciones diferentes y muestra algunos ejemplos para su revisión. Esto permite examinar situaciones ambiguas, expresiones negativas indirectas, comentarios neutrales o diferencias entre el sentimiento del texto y la calificación numérica. El análisis también puede ayudar a detectar errores en las etiquetas originales o identificar casos en los que sería conveniente adaptar un modelo mediante fine-tuning.

	comentario	aspecto	calificacion	label_real	Predicción - RoBERTuito	Predicción - DistilBERT multilingüe	Predicción - XLM-RoBERTa
18	La comida fue aceptable, podría mejorar. (menc...	comida	3	NEU	POS	POS	NEU
20	La servicio estuvo regular, cumplió sin destac...	servicio	3	NEU	POS	NEU	NEU
50	La servicio estuvo regular, cumplió sin destac...	servicio	3	NEU	POS	NEU	NEU
64	La comida fue aceptable, podría mejorar. (menc...	comida	3	NEU	POS	POS	NEU
70	La ambiente estuvo regular, cumplió sin destac...	ambiente	3	NEU	POS	NEU	NEG
75	La ambiente estuvo bien, nada fuera de lo norm...	ambiente	3	NEU	POS	POS	NEU
80	La comida fue aceptable, podría mejorar. (menc...	comida	3	NEU	POS	POS	NEU
83	La comida fue aceptable, podría mejorar. (menc...	comida	3	NEU	POS	POS	NEU
97	La comida fue aceptable, podría mejorar. (menc...	comida	3	NEU	POS	POS	NEU
98	La comida estuvo regular, cumplió sin destacar...	comida	3	NEU	POS	NEU	NEU
101	La comida estuvo regular, cumplió sin destacar...	comida	3	NEU	POS	NEU	NEU
102	La comida fue aceptable, podría mejorar. (menc...	comida	3	NEU	POS	POS	NEU
107	La comida estuvo regular, cumplió sin destacar...	comida	3	NEU	POS	NEU	NEU
111	La ambiente estuvo regular, cumplió sin destac...	ambiente	3	NEU	POS	NEU	NEG
118	La comida estuvo regular, cumplió sin destacar...	comida	3	NEU	POS	NEU	NEU
134	La comida fue aceptable, podría mejorar. (menc...	comida	3	NEU	POS	POS	NEU
142	La comida fue aceptable, podría mejorar. (menc...	comida	3	NEU	POS	POS	NEU
143	La comida fue aceptable, podría mejorar. (menc...	comida	3	NEU	POS	POS	NEU
148	La comida estuvo regular, cumplió sin destacar...	comida	3	NEU	POS	NEU	NEU
155	La comida fue aceptable, podría mejorar. (menc...	comida	3	NEU	POS	POS	NEU

Imagen 15.-Selección del mejor modelo en base a las métricas



Resultados

Los resultados obtenidos demostraron que los tres modelos fueron capaces de clasificar correctamente las opiniones del conjunto de datos en categorías positivas, negativas y neutrales, aunque con diferencias en su desempeño. El análisis se apoyó en métricas ampliamente utilizadas en problemas de clasificación, permitiendo medir tanto la exactitud global como la capacidad de identificar correctamente cada tipo de sentimiento. Las tablas comparativas, informes detallados y matrices de confusión facilitaron la identificación de fortalezas y debilidades específicas de cada modelo.

La comparación reveló que algunos modelos mostraron un mejor equilibrio entre precisión y capacidad de recuperación de clases, mientras que otros destacaron por una mayor velocidad de procesamiento. Las matrices de confusión evidenciaron que las mayores dificultades se presentaron en la clasificación de opiniones neutrales, debido a que este tipo de comentarios suele contener expresiones ambiguas o menos explícitas que las opiniones claramente positivas o negativas. Además, las gráficas comparativas permitieron observar de manera visual las diferencias de rendimiento y eficiencia entre las alternativas evaluadas. Finalmente, mediante la combinación de métricas de calidad y tiempos de inferencia, se determinó el modelo con mejor desempeño general, considerando tanto la efectividad de clasificación como los requerimientos de procesamiento.

Si bien un modelo destacó de los demás, es posible que alguno de los otros 2 modelos se destaque mejor con otros data sets, al final la resolución de cada problema dependerá de la cantidad y calidad de los datos, si obtenemos reseñas muy ambiguas o neutras el modelo podría determinar como una reseña positiva o negativa y terminar en ser un caso de positivo negativo o falso negativo.



Conclusiones

La evaluación comparativa de modelos preentrenados de Hugging Face permitió comprobar que las técnicas modernas de procesamiento de lenguaje natural representan una alternativa eficaz para automatizar el análisis de sentimientos en opiniones de restaurantes. El uso de modelos previamente entrenados redujo significativamente el esfuerzo de desarrollo, permitiendo obtener resultados útiles desde las primeras etapas del proyecto. Asimismo, la aplicación de métricas de evaluación y herramientas visuales facilitó una comparación objetiva entre las diferentes soluciones analizadas.

Los resultados obtenidos demuestran que la selección de un modelo no debe basarse únicamente en la precisión alcanzada, sino también en aspectos relacionados con la eficiencia computacional y las necesidades específicas del problema de negocio. Mientras algunos escenarios pueden priorizar la velocidad de respuesta, otros requieren una mayor capacidad para identificar correctamente opiniones negativas o ambiguas. En conclusión, la metodología aplicada permitió identificar la alternativa más adecuada para el conjunto de datos utilizado y proporcionó una base sólida para futuras mejoras, como el ajuste fino de modelos especializados o la incorporación de nuevos conjuntos de datos que permitan incrementar aún más el desempeño del sistema de análisis de sentimientos.

En la práctica la resolución de cada caso real dependerá de la cantidad y calidad de los datos, así como el objetivo al que se quiera llegar, saber utilizar estos modelos como base podría resultar en ofrecer herramientas poderosas que no se limiten únicamente a la evaluación de reseñas de un restaurante, si no se extienda a más recolección de datos de opiniones/reseñas en diferentes ámbitos.

